

# TEMELDE PYTHON PROGRAMI NASIL GÖRÜNÜR

[Kaynak](#)

## İÇERİK

1. Python Programlama Temelleri: Değişkenler, STDIN İşleme ve Veri Tipleri
2. Conditional Statements (Koşullu İfadeler) ve Kontrol Akışı
3. Iterations (Döngüler) ve Sürekli Yürütme

## Python Programlama Temelleri: Değişkenler, STDIN İşleme ve Veri Tipleri

Bir otomasyon veya siber güvenlik aracı (script) geliştirirken, uygulamanın o anki durumunu (state) hafızada tutmak ve kullanıcıdan alınan verileri işlemek temel bir gereksinimdir. Bu notta, bir "Sayı Tahmin Oyunu"nun ilk aşamasını inşa ederken değişken (variable) yönetimini, kütüphane içe aktarmayı ve veri tipi dönüşümlerini (Type Casting) teknik bir perspektifle inceliyoruz.

## Rastgele Sayı Üretimi ve Değişken (Variable) Atamaları

Programın temel mantığı, sistemin belirli bir aralıkta gizli bir sayı belirlemesi ve kullanıcının bu sayıyı tahmin etmesidir. Bu durum, arka planda farklı veri parçalarını hafızada tutmamızı gerektirir.

Python'da rastgele değerler üretmek için yerleşik bir kütüphane olan `random` kütüphanesini kullanırız. Bu kütüphane içerisindeki `random.randint(a, b)` metodu, belirtilen `a` ve `b` sınırları dahil olmak üzere rastgele bir tam sayı (Integer) döndürür.

```
import random # Rastgele sayı üretimi için gerekli araçları barındıran
kütüphaneyi içe aktarır.

secret = random.randint(1, 20) # 1 ile 20 arasında (1 ve 20 dahil) rastgele
bir tam sayı üretir ve 'secret' değişkenine atar.
tries = 0
guess = 0 # Tahmin değerini, geçerli aralığın (1-20) dışında bir değere
eşitliyoruz.

print("I'm thinking of a number between 1 and 20")
```

Yukarıdaki kod bloğunun arka plandaki teknik mantığı şu şekildedir: Öncelikle **secret** değişkeni ile programın hedef sayısını hafızaya alıyoruz. Kullanıcının kaç deneme yaptığını takip etmek (State Tracking - Durum Takibi) amacıyla **tries** adında bir sayaç değişkeni oluşturuyor ve bunu **0** olarak başlatıyoruz.

**ÖNEMLİ:** `guess` (tahmin) değişkenine başlangıç değeri olarak özellikle **0** atanmıştır. Eğer bu değişkene 1 ile 20 arasında bir başlangıç değeri verseydik ve bu değer tesadüfen **secret** değişkeni ile aynı olsaydı, program mantıksal bir hataya (Logical Bug) düşer ve kullanıcı henüz girdi yapmadan oyunu kazanmış sayılırdı. Başlangıç değerinin geçerli aralık dışında (Out of Bounds) tutulması, bu tür senaryoları engellemek için kritik bir programlama pratiğidir. Kodun sonunda yer alan **print()** metodu ise Standart Çıktı (STDOUT) üzerinden kullanıcıyı bilgilendirmek için kullanılmıştır.

## Girdi Yakalama (Input Handling) ve Tip Dönüşümü (Type Casting)

Değişkenlerimizi hazırladıktan sonra, programın dış dünyadan, yani kullanıcıdan bir veri alması gerekir. Python'da Standart Girdi'den (STDIN) veri okumak için **input()** fonksiyonu kullanılır.

```
text = input("Take a guess: ") # STDIN'den veri okur ve her zaman metin
(String) formatında döndürür.
guess = int(text) # Metin tipindeki veriyi sayısal bir tam sayı (Integer)
tipine dönüştürür.

tries = tries + 1 # Deneme sayacını 1 artırır (tries += 1 şeklinde de
yazılabilir).
```

Bu adımda siber güvenlik scriptleri yazarken de sıklıkla karşılaşılan çok kritik bir teknik detay devreye girmektedir: **Type Casting (Veri Tipi Dönüştürme)**. **input()** fonksiyonu, kullanıcı terminale bir rakam (örneğin 15) girmiş olsa bile, bu veriyi hafızaya her zaman karakter dizisi yani **String ("15")** olarak kaydeder. Programın ilerleyen aşamalarında kullanıcının girdiği bu değer ile **secret** (ki bu bir Integer'dır) değerini matematiksel olarak karşılaştırmamız gerekecektir. Bir String ile bir Integer doğrudan büyüklük/küçüklük veya matematiksel eşitlik testine sokulamaz; bu durum tip hatasına (TypeError) yol açar.

Bu hatayı önlemek için **int()** fonksiyonunu kullanarak, **text** değişkeni içerisindeki String veriyi sayısal bir formata (Integer) dönüştürüyor (Type Casting) ve asıl **guess** değişkenimize atıyoruz. Ardından kullanıcının bir eylem gerçekleştirdiğini kayıt altına almak için **tries** sayacımızı 1 artırıyoruz.

## Taslak Kodun Bütünleştirilmesi ve Mevcut Sınırlamalar

Geliştirdiğimiz bu yapısal temelleri birleştirdiğimizde, **guess\_v1.py** adıyla **/home/ubuntu/Python-Demo** dizininde saklanan aşağıdaki script ortaya çıkmaktadır.

```
import random # Rastgele sayı üretimi için araçlar sağlar.

secret = random.randint(1, 20) # a <= secret <= b aralığında sayı belirler.
tries = 0
guess = 0 # secret 1-20 arasında olduğu için, hedef olamayacak bir değerle
```

başlar.

```
print("I'm thinking of a number between 1 and 20")
```

```
text = input("Take a guess: ") # input() veriyi metin (string) olarak döndürür.
```

```
guess = int(text) # Karşılaştırma yapabilmek için metni tam sayıya (integer) çevirir.
```

```
tries = tries + 1 # Deneme sayısını kaydeder.
```

**DİKKAT:** Bu kod bloğu, değişken atamaları, kütüphane kullanımı ve kullanıcıdan güvenli/doğru veri tipiyle girdi alma konularında kusursuz çalışmasına rağmen, şu anki haliyle bir oyun veya mantıksal bir uygulama (Execution) aşamasında eksiktir. Kod, verileri almakta ancak bu verileri işlememektedir. Yani **guess** ile **secret** değişkenleri arasında herhangi bir kontrol veya karşılaştırma yapılmamaktadır. Bu betiğin (script) amacına ulaşması için bir sonraki aşamada Şartlı İfadelerin (Conditional Statements - If/Else) ve Karşılaştırma Operatörlerinin (Comparison Operators) sürece dahil edilmesi, kontrol akışının (Control Flow) sağlanması gerekmektedir.

## Python Programlama Temelleri: Conditional Statements (Koşullu İfadeler) ve Kontrol Akışı

Bir önceki aşamada hafızaya aldığımız (State Tracking) ve doğru veri tipine dönüştürdüğümüz (Type Casting) kullanıcı girdisi, kendi başına hiçbir anlam ifade etmez. Bir programın veya siber güvenlik betiğinin (script) zekası, verileri birbirleriyle karşılaştırıp ortaya çıkan sonuca göre farklı yollar izleyebilmesinde yatar. Bu kavrama programlamada **Control Flow (Kontrol Akışı)** denir ve Python'da bu akışı yönlendirmek için **Conditional Statements (Koşullu İfadeler)** kullanılır.

Aşağıdaki kod bloğunda, kullanıcının tahmini ( `guess` ) ile sistemin belirlediği hedef değeri ( `secret` ) karşılaştırmak ve kullanıcıya geri bildirim (feedback) vermek için oluşturduğumuz koşul yapısını inceleyeceğiz.

```
# Give a hint using if / elif / else.
if guess < 1 or guess > 20:
    print("That number is out of range. Try again.")
elif guess < secret:
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it in", tries, "tries!")
```

Bu koşul bloğunun arka plandaki teknik ve mantıksal işleyişi şu şekildedir:

Sistem, koşulları her zaman yukarıdan aşağıya doğru sıralı bir şekilde (Sequentially) değerlendirir. Her bir ifade arka planda bir **Boolean Logic (Mantıksal Doğruluk)** testi yapar (True veya False).

1. `if guess < 1 or guess > 20:` : Programın kontrol ettiği ilk güvenlik ve mantık bariyeridir. Burada **or (veya)** mantıksal operatörü kullanılmıştır. Eğer kullanıcı 1'den küçük VEYA 20'den büyük bir değer girerse, bu ifade `True` (Doğru) değerini döndürür. Program "Out of range" (Kapsam dışı) uyarısını basar ve bloğun geri kalanını tamamen görmezden gelir. Sınır değerlerinin (Out of Bounds) en başta kontrol edilmesi, uygulamanın beklenmeyen girdilerle (Unexpected Input) hatalı işlem yapmasını engeller.
2. `elif guess < secret:` : `elif` (Else If), yalnızca bir üstündeki koşul `False` (Yanlış) döndürdüğünde devreye girer. Yani kullanıcının girdisi geçerli aralıkta (1-20) ise program bu satıra ulaşır ve tahminin hedef sayıdan küçük olup olmadığını kontrol eder.
3. `elif guess > secret:` : Eğer sayı küçük de değilse, program bu kez tahminin hedeften büyük olup olmadığını kontrol eder.
4. `else:` : Burası koşul zincirinin son durağıdır (Fallback). Eğer tahmin geçerli aralıktaysa, hedeften küçük değilse ve hedeften büyük de değilse; geriye kalan tek matematiksel olasılık hedefe **eşit** olmasıdır. Bu yüzden tekrar bir `elif guess == secret:` yazarak işlemciyi yormaya gerek yoktur. Program `else` bloğuna düşer ve kullanıcının kaç denemede (tries) başarılı olduğunu STDOUT'a basar.

**ÖNEMLİ:** Python'da `if/elif/else` yapılarında, koşullardan herhangi biri `True` olduğu anda program o bloğun içindeki komutları çalıştırır ve koşul zincirinin (Condition Chain) geri kalanındaki hiçbir satırı okumaz (Execution Path - Çalışma Yolu sonlanır). Bu yapı, kaynak tüketimini optimize eder.

## Sayısal Senaryolar Üzerinden Execution Path (Çalışma Yolu) Analizi

Sistemin arka planda gizli sayıyı (**secret**) **10** olarak belirlediğini varsayarak, farklı kullanıcı girdilerinin program akışını nasıl değiştirdiğini analiz edelim:

- **Senaryo 1 (Girdi: 30):** 30 sayısı 20'den büyük olduğu için ilk `if` koşulu (`guess > 20`) `True` döner. Program doğrudan kapsam dışı hatasını basar. Diğer hiçbir `elif` kontrol edilmez.
- **Senaryo 2 (Girdi: 5):** 5 sayısı sınırların içindedir, bu yüzden ilk `if` koşulu `False` döner. Akış bir sonraki adıma geçer. İlk `elif` bloğunda `5 < 10` ifadesi sınanır. Bu `True` olduğu için program ekrana "Too low" (Çok düşük) yazar ve zinciri sonlandırır.
- **Senaryo 3 (Girdi: 15):** İlk `if` bloğu `False` döner. İlk `elif` (`15 < 10`) bloğu da `False` döner. Akış ikinci `elif` bloğuna geçer. `15 > 10` koşulu `True` döner, program ekrana "Too high" (Çok yüksek) yazar ve işlemi bitirir.

## İlk Taslağın Tamamlanması (guess\_v2.py)

Yukarıda öğrendiğimiz koşullu kontrol mekanizmasını bir önceki notta oluşturduğumuz değişken yapısına entegre ettiğimizde, hedef sistemimizdeki **/home/ubuntu/Python-Demo** dizininde bulunan **guess\_v2.py** dosyasının son hali ortaya çıkar.

```
import random # Rastgele sayı üretimi için kütüphane içe aktarımı.

secret = random.randint(1, 20) # a <= secret <= b mantığıyla 1-20 arası
gizli sayı üretilir.
tries = 0
guess = 0 # Başlangıç değeri (Initial State) olarak geçersiz bir değer (0)
atanır.

print("I'm thinking of a number between 1 and 20")

text = input("Take a guess: ") # STDIN'den veri string olarak alınır.
guess = int(text) # Karşılaştırma yapabilmek için string veri integer'a
dönüştürülür (Type Casting).

tries = tries + 1 # Kullanıcı hamle yaptığı için sayaç güncellenir.

# if / elif / else yapılarıyla Mantıksal Karşılaştırma (Logical Comparison)
ve Geri Bildirim
if guess < 1 or guess > 20:
    print("That number is out of range. Try again.")
elif guess < secret:
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it in", tries, "tries!")
```

**DİKKAT:** Bu kod, kullanıcının girdisini mantıksal olarak test edebilen, tamamen işlevsel (Functional) bir programdır. Çalışma dosyalarının yer aldığı arşiv (Zip) dosyası üzerinden bu evrimi kendi lokal sisteminizde test edebilirsiniz. Ancak sistemdeki çok büyük bir teknik limitasyon (Limitation) hala devam etmektedir: Program yukarıdan aşağıya bir kez çalışır (Single Execution), koşulu değerlendirir, sonucu basar ve **kapanır**. Kullanıcıya oyunu kazanana kadar tekrar tekrar deneme şansı (Sonsuz Tries) sunabilmek için, akışın sadece ileri doğru gitmesi değil, belirli koşullar sağlanana kadar kendi içinde dönmesi gereklidir. Bu da bizi bir sonraki konsept olan Döngüler (Loops) mantığına götürecektir.

## Python Programlama Temelleri: Iterations (Döngüler) ve Sürekli Yürütme

Siber güvenlik araçlarında ve genel programlamada, bir işlemin belirli bir başarı veya durum koşulu sağlanana kadar otonom olarak tekrar etmesi için **Iterations (Döngüler)** kullanılır.

Önceki taslağımızdaki "tek seferlik çalışma ve kapanma" limitasyonunu çözmek adına, kontrol akışımıza bir **while loop (while döngüsü)** entegre ediyoruz.

Aşağıda, hedef sistemimizdeki **/home/ubuntu/Python-Demo** dizininde yer alan ve uygulamanın nihai halini temsil eden **guess\_v3.py** kodunun temel çalışma mantığı özetlenmiştir.

```
import random # Rastgele sayı üretimi için gerekli kütüphane

secret = random.randint(1, 20) # a <= secret <= b sınırlarında hedef sayıyı belirle
tries = 0
guess = 0 # Başlangıç durumu (Initial State)

print("I'm thinking of a number between 1 and 20")

# Ana Döngü: Tahmin, gizli sayıya eşit olmadığı sürece (guess != secret)
# aşağıdaki veri alma ve karşılaştırma bloğunu tekrarla.
while guess != secret:
    text = input("Take a guess: ") # STDIN üzerinden veriyi oku
    guess = int(text) # Karşılaştırma için Type Casting (String -> Integer)
    uygula

    tries = tries + 1 # Sayaç (State Tracking) güncelle

# Koşullu Kontrol Akışı (Control Flow)
if guess < 1 or guess > 20:
    print("That number is out of range. Try again.")
elif guess < secret:
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it in", tries, "tries!")
```

**ÖNEMLİ:** Buradaki mantıksal döngünün temelinde, **!= (Not Equal - Eşit Değildir)** operatörü ile kurulan durum kontrolü yatmaktadır. Program her iterasyonun başlangıcında **guess != secret** ifadesinin mantıksal durumunu (Boolean True/False) sınar. Kullanıcı her yanlış tahmin yaptığında koşul **True** döner; döngü bloğu yeniden çalıştırılarak kullanıcıya yeni bir deneme hakkı sunulur. Kullanıcı doğru sayıyı girdiğinde ise bu koşul matematiksel olarak **False** (Örn: **10 != 10** ifadesi yanlıştır) sonucunu üretir. Bu noktada döngü kırılarak sonlandırılır ve programın çalışma zamanı (Execution) başarıyla tamamlanmış olur.